Deliverable — The Real-World Target (T1)

The job: pick a real, recently-broken piece of Android software to point our LLM agent at, and reason about whether its bug still works once the hardened allocator (Scudo + MTE) is turned on. We landed on using **two** targets, one of each bug shape.

Word bank (the terms in this tab)

Term	Plain meaning
CVE	A public ID for one specific known bug (e.g. CVE-2025-48592).
Heap	The pool of memory a program asks for at runtime with `malloc`. Where these bugs live.
Out-of-bounds (OOB)	Reading or writing past the end of a buffer — into memory that wasn't yours.
OOB **read**	You *read* memory you shouldn't → leaks secrets. Lower stakes.
OOB **write**	You *write* memory you shouldn't → corrupts other data → can take over the program. Higher stakes.
Zero-click	No user action needed. A malicious file/message triggers it on arrival.
Privilege escalation	Going from "just an app" to "controls the system."
Codec / decoder	Code that turns a compressed file (video, image) back into pixels. A classic place for parsing bugs.
Granule	The 16-byte chunk MTE colors as a unit. An overflow staying inside one granule is invisible to MTE.
Spatial vs temporal	Spatial = out-of-bounds (wrong *place*). Temporal = use-after-free (wrong *time*, memory already freed).

Target A — dav1d (CVE-2025-48592): the READ target

What it is: a heap out-of-bounds **read** in `C2SoftDav1dDec.cpp`, Android's wrapper around **dav1d** (the AV1 video decoder). A crafted video makes the decoder read past a buffer and leak memory. Remote, zero-click, Android 15/16, patched Dec 2025, CVSS 7.5.

Why it's a good target:

- It's C/C++ on ARM, runs on Scudo — exactly our setup.
- Remote and zero-click: the strongest, most realistic attack surface.
- It's one small, self-contained decoder, so we can **build it and reproduce the bug in an afternoon** — ideal for getting our test harness working.
- There's a patch commit to compare against, so we can see the bug precisely.

Does the bug survive MTE? (the reasoning Marius asked for): probably not, and that's fine. It's a *read*. With MTE on, the moment the read crosses into the neighboring allocation (which has a different color), the CPU **crashes the program** about 15 times out of 16. So the leak most likely turns into "the app just dies." To still pull it off, the attacker would need to *learn the color first* — and this is a remote video parser, the hardest place to do that. **"MTE probably wins here" is itself a clean, publishable result.**

Drawback (state it openly): because it's only a read, "success" is weak (a leak, not control), and it measures MTE at its **strongest**. Good for proving the harness; less dramatic as a finding.

Target B — Skia (CVE-2025-32318): the WRITE target

What it is: a heap out-of-bounds **write** in **Skia**, Android's 2D graphics library. Crafted graphics input writes past a buffer and corrupts adjacent memory, leading to **privilege escalation**. Remote, no user interaction, Android 16, CVSS 8.8 (impact rated "total"). No public PoC.

Why it's worth adding:

- It's a **write**, so "success" is a real security effect (taking control), not just a leak. This is the path MTE is actually designed to stop, so "can the agent get around it" is a meatier question.
- Skia processes untrusted graphics everywhere, so it's a genuinely important surface. Also C++ on ARM, on Scudo.

Drawbacks (state them openly):

- It's privilege-escalation-flavored, which sits nearer MTE's **weaker** end (where leaking the color is easier), so a win here is less impressive than beating MTE at its strong end.
- Skia is a **big** codebase — much harder to build and isolate than one small decoder. No public PoC or vulnerable-code detail yet.
- **Open question for Marius:** is the bug reached by *remote* rendered content, or only *after* the attacker already has a foothold? That answer decides whether Skia tests MTE at its strong or weak end.

Why two targets, and in what order

One read + one write covers both halves of what MTE does (catching bad reads *and* bad writes), and lets us say something general instead of a single data point. Admitting each one's drawback is part of what makes the write-up trustworthy.

Order: dav1d first. It's the fastest to stand up, so it de-risks the harness and success-checker. Only once that machinery is proven do we add Skia for the write case. **Not in parallel** — that's how a two-target project stalls into two half-finished ones.

Backups (if either lead proves hard to build)

CVE	Component	Shape	Note
CVE-2024-5171	libaom	integer → heap overflow	Another clean, standalone C library (AV1 encoder side). Fallback for the read/overflow slot.
CVE-2025-0084 / 0075 / 0074	Android Bluetooth SDP	use-after-free	UAFs test *temporal* detection (wrong-time), a third flavor. Messier to isolate than a codec.

Avoid as a first target: Chrome CVEs (Skia-in-Chrome, Downloads, Media) — Chrome mostly uses its own allocator (**PartitionAlloc**), not Scudo, so they don't test the allocator Marius named. Kernel-driver CVEs are local privilege escalation, where MTE is weakest.

Honest caveat: I've reasoned from the CVE write-ups, not from building either target or running a PoC. Actually building dav1d, reproducing the bug without MTE, then turning MTE on — that's Phase 0 of the plan, and the first real step.